

Recommended approach

Use the **official Google Play In-App Updates API**, consumed in Flutter through the maintained wrapper package `in_app_update` (currently v4.2.5, wraps `com.google.android.play:app-update / app-update-ktx` 2.1.0). This needs no backend, no hardcoded "latest version" string, and no scraping of the Play Store — Play itself tells your app whether the installed build is behind, and the popup naturally stops appearing once the user updates because the answer comes from Play, not from a flag you maintain.

Ranked options:

1. **Play In-App Update API only** (recommended, default for this app) — official, free, zero backend, automatically respects staged/partial rollouts.
2. **Play In-App Update API + a free remote "force update" switch** (Firebase Remote Config free tier, or any tiny JSON endpoint you already host) — add this only if you need a hard kill-switch for a specific version independent of Play Console's own priority/rollout timing (e.g., a critical break-glass fix).
3. **Manually comparing a hardcoded/backend version number against** `package_info_plus` — workable in a pinch, but it duplicates what Play already gives you for free, it goes stale the moment you forget to bump it, and it ignores staged rollouts (it'll nag users into installing a build they aren't even eligible for yet). Use this only as a fallback for stores other than Google Play.
4. **Scraping the Play Store page / unofficial endpoints** — explicitly avoid. Fragile, against Play's terms, no official support.

B. Why this approach is best

Play's `AppUpdateManager` already knows: your installed version code, the latest version code for *your specific user* (accounting for staged rollout eligibility), whether the user already declined/has the update mid-flight, and the developer-set priority of the release. Re-implementing any of that with your own version checks means re-solving problems Google already solved, and it'll actively misbehave during staged rollouts (you'd nag a user Play hasn't even offered the update to). The API also gives you two UX patterns out of the box — a blocking full-screen **Immediate** update and a background-download **Flexible** update with its own restart prompt — both built and tested by Google, with zero billing involved (`app-update / app-update-ktx` is part of the free Play Core libraries, no Play Console seat or paid quota required).

C. Step-by-step implementation

1. Add the package: `in_app_update: ^4.2.5` to `pubspec.yaml`, then `flutter pub get`.

2. Confirm `android/app/build.gradle`'s `minSdkVersion` is 21+ (required by the plugin; Flutter's default is already fine for almost every project).
3. No manifest changes, no new permissions — it talks to the Play Store app already on the device.
4. Create a single `AppUpdateService` (GetX service) that wraps `checkForUpdate()`, decides immediate vs. flexible, and exposes one entry point: `checkAndHandleUpdate()`.
5. Register it once, early, as a permanent Get service (in `main()` or your `InitialBinding`).
6. Call `checkAndHandleUpdate()` from your splash screen / bootstrap controller before navigating into the app, and again whenever the app resumes from background (to catch a stalled immediate update or a flexible update that finished downloading while the app was backgrounded).
7. Branch on the result: no-op if up to date, resume a stalled immediate update, launch a new immediate update for high-priority/stale releases, or offer a dismissible flexible-update prompt otherwise.
8. Wrap every call in try/catch — non-Play installs, no network, and Play Store being unavailable all surface as exceptions, not as "no update," and must fail silently.
9. Ship a signed release build to an Internal Testing (or Internal App Sharing) track to actually test this — `flutter run` debug builds will throw `ERROR_API_NOT_AVAILABLE` every time, since the API only works for Play-installed builds.

D. Exact code examples

pubspec.yaml

```
dependencies:  
  in_app_update: ^4.2.5  
  get_storage: ^2.1.1 # you likely already have this with GetX
```

lib/app/services/app_update_service.dart

```
import 'package:flutter/widgets.dart';  
import 'package:get/get.dart';  
import 'package:get_storage/get_storage.dart';  
import 'package:in_app_update/in_app_update.dart';  
  
/// Centralizes all Google Play in-app update logic.  
/// Lives for the app's lifetime; registered once, called from splash +  
onResume.  
class AppUpdateService extends GetxService with WidgetsBindingObserver {  
  final _box = GetStorage();  
  static const _kFlexSnoozeKey = 'update_flex_snoozed_until';  
}
```

```

// Tune these to your release strategy / Play Console "inAppUpdatePriority".
static const _highPriorityThreshold = 4; // 0-5 scale set in Play Console
static const _minStalenessDaysForFlexible = 0;

bool _checking = false;

@override
void onInit() {
    super.onInit();
    WidgetsBinding.instance.addObserver(this);
}

@override
void onClose() {
    WidgetsBinding.instance.removeObserver(this);
    super.onClose();
}

@override
void didChangeAppLifecycleState(AppLifecycleState state) {
    // Re-check whenever the app comes back to the foreground:
    // catches a stalled immediate update, or a flexible update
    // that finished downloading while the app was backgrounded.
    if (state == AppLifecycleState.resumed) {
        checkAndHandleUpdate();
    }
}

/// Single entry point. Call from splash on startup, and on resume.
Future<void> checkAndHandleUpdate() async {
    if (_checking) return; // avoid overlapping checks
    _checking = true;
    try {
        final info = await InAppUpdate.checkForUpdate();
        await _route(info);
    } catch (_) {
        // Network down, Play Store unavailable, app not installed via Play
        // (sideloaded/debug/F-Droid/emulator) -> stay silent, never block the
app.
    } finally {
        _checking = false;
    }
}

Future<void> _route(AppUpdateInfo info) async {
    switch (info.updateAvailability) {

```

```

        case UpdateAvailability.unknown:
        case UpdateAvailability.updateNotAvailable:
            return; // Up to date -> nothing to show, nothing to remember.

        case UpdateAvailability.developerTriggeredUpdateInProgress:
            // An immediate update was already accepted but got interrupted
            // (app killed mid-update). Resume it silently.
            if (info.immediateUpdateAllowed) {
                await InAppUpdate.performImmediateUpdate();
            }
            return;

        case UpdateAvailability.updateAvailable:
            final isCritical = info.updatePriority >= _highPriorityThreshold;

            if (isCritical && info.immediateUpdateAllowed) {
                await _runImmediateUpdate();
            } else if (info.flexibleUpdateAllowed) {
                await _maybeOfferFlexibleUpdate(info);
            }
            return;
    }
}

Future<void> _runImmediateUpdate() async {
    try {
        await InAppUpdate.performImmediateUpdate();
        // Play restarts the app on success; if this returns, the user
        // canceled or it failed -- just let them continue using the app.
    } catch (_) {
        // Canceled / failed: don't loop forever, just try again next launch.
    }
}

Future<void> _maybeOfferFlexibleUpdate(AppUpdateInfo info) async {
    // Cosmetic throttle only -- NOT what determines whether an update
    // exists. Prevents re-nagging the same user every cold start.
    final snoozedUntil = _box.read<int>(_kFlexSnoozeKey) ?? 0;
    if (DateTime.now().millisecondsSinceEpoch < snoozedUntil) return;
    if ((info.clientVersionStalenessDays ?? 0) < _minStalenessDaysForFlexible)
{
        return;
    }

    final accepted = await Get.dialog<bool>(
        AlertDialog(

```

```

        title: const Text('Update available'),
        content: const Text(
            'A new version of the app is ready. Update now for the latest
features and fixes.',
        ),
        actions: [
            TextButton(
                onPressed: () => Get.back(result: false),
                child: const Text('Later'),
            ),
            TextButton(
                onPressed: () => Get.back(result: true),
                child: const Text('Update'),
            ),
        ],
    ),
    barrierDismissible: false,
);

if (accepted != true) {
    // Snooze for 24h so it doesn't reappear on every relaunch this session.
    _box.write(
        _kFlexSnoozeKey,
        DateTime.now().add(const Duration(hours: 24)).millisecondsSinceEpoch,
    );
    return;
}

try {
    await InAppUpdate.startFlexibleUpdate(); // downloads in background
    _showRestartSnackbar(); // completes once downloaded
} catch (_) {
    // User canceled the consent dialog or download failed.
}
}

void _showRestartSnackbar() {
    Get.snackbar(
        'Update ready',
        'Restart to finish installing the update.',
        isDismissible: false,
        duration: const Duration(days: 1), // stays until acted on
        mainButton: TextButton(
            onPressed: () => InAppUpdate.completeFlexibleUpdate(),
            child: const Text('RESTART', style: TextStyle(color: Colors.white)),
        ),
    ),

```

```

    );
  }
}

```

Registering it (main.dart)

```

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  GetStorage.init();
  Get.put(AppUpdateService(), permanent: true);
  runApp(const MyApp());
}

```

Splash controller

```

class SplashController extends GetxController {
  @override
  void onReady() async {
    super.onReady();
    await Get.find<AppUpdateService>().checkAndHandleUpdate();
    Get.offAllNamed(Routes.HOME);
  }
}

```

For reference — what this plugin does natively (Kotlin), since you asked for the underlying API:

```

val appUpdateManager = AppUpdateManagerFactory.create(context)
appUpdateManager.appUpdateInfo.addOnSuccessListener { info ->
    if (info.updateAvailability() == UpdateAvailability.UPDATE_AVAILABLE
        && info.updatePriority() >= 4
        && info.isUpdateTypeAllowed(AppUpdateType.IMMEDIATE))
    {
        appUpdateManager.startUpdateFlowForResult(
            info, activityResultLauncher,
            AppUpdateOptions.newBuilder(AppUpdateType.IMMEDIATE).build()
        )
    }
}
}

```

This is exactly what `in_app_update`'s Android channel implementation calls under the hood — you don't need to touch this directly unless you ever drop Flutter for a native module.

E. Validation logic for "show once only when outdated"

The key insight: correctness of "is this user outdated" comes entirely from

`InAppUpdate.checkForUpdate()` on every check — you never need to persist "already prompted because outdated," because the moment the user installs the update, Play's own answer flips to `updateNotAvailable` and the code above returns immediately, doing nothing. The only thing worth caching locally is *UX throttling* of the optional flexible dialog (the 24h snooze above), never the underlying "is there an update" fact.

Flow per check:

1. `checkForUpdate()` throws → silently stop (no popup).
2. `updateNotAvailable` or `unknown` → silently stop (no popup) — this is what happens forever after the user updates.
3. `developerTriggeredUpdateInProgress` → resume the stalled immediate flow, no new dialog.
4. `updateAvailable` + high priority → immediate (blocking) update, no snooze logic at all — it should reappear every launch until installed.
5. `updateAvailable` + normal priority → flexible dialog, respecting the 24h local snooze; once downloaded, the "Restart" snackbar persists until the user acts (don't snooze that one).

F. Common mistakes to avoid

Testing with `flutter run /debug` APKs and concluding the integration is broken — it only works for builds installed through Play (use Internal Testing or Internal App Sharing). Re-checking on every navigation/rebuild instead of once on cold start plus on resume, which causes the dialog to reappear mid-session. Forgetting the `onResume` check entirely, which leaves an immediate update permanently stalled if the app was killed mid-flow, or leaves a downloaded flexible update silently occupying device storage forever because the restart prompt never got a chance to fire again. Hardcoding a "latest version" string in Dart code to compare against — it goes stale the day after release and ignores staged rollouts. Treating a staged/partial rollout as a bug ("why doesn't the popup show for everyone") — that's Play intentionally not yet offering the update to that user; nothing to fix. Not catching exceptions, which crashes the app on sideloaded installs, emulators, or alternative stores where the Play API simply isn't available.

G. Final recommended folder/file placement

```
lib/  
  app/  
    services/  
      app_update_service.dart      <- new file (section D)  
  modules/  
    splash/
```

```
controllers/
  splash_controller.dart    <- calls checkAndHandleUpdate()
main.dart                  <- Get.put(AppUpdateService(), permanent:
true)
pubspec.yaml               <- add in_app_update dependency
android/app/build.gradle   <- verify minSdkVersion >= 21
```

H. Minimal patch plan

Add `in_app_update: ^4.2.5` to `pubspec.yaml` and run `flutter pub get`. Create `lib/app/services/app_update_service.dart` with the class from section D. In `main()`, call `Get.put(AppUpdateService(), permanent: true)` before `runApp`. In your existing splash/bootstrap controller's `onReady`, await `Get.find<AppUpdateService>().checkAndHandleUpdate()` before navigating to the home route. No manifest or Gradle changes are needed beyond confirming `minSdkVersion`. To verify it actually works, build a signed AAB, upload it to an Internal Testing track in Play Console, install the app from that track link on a device, then publish a second build with a higher version code — the prompt should appear within minutes once Play registers the new release for that track.